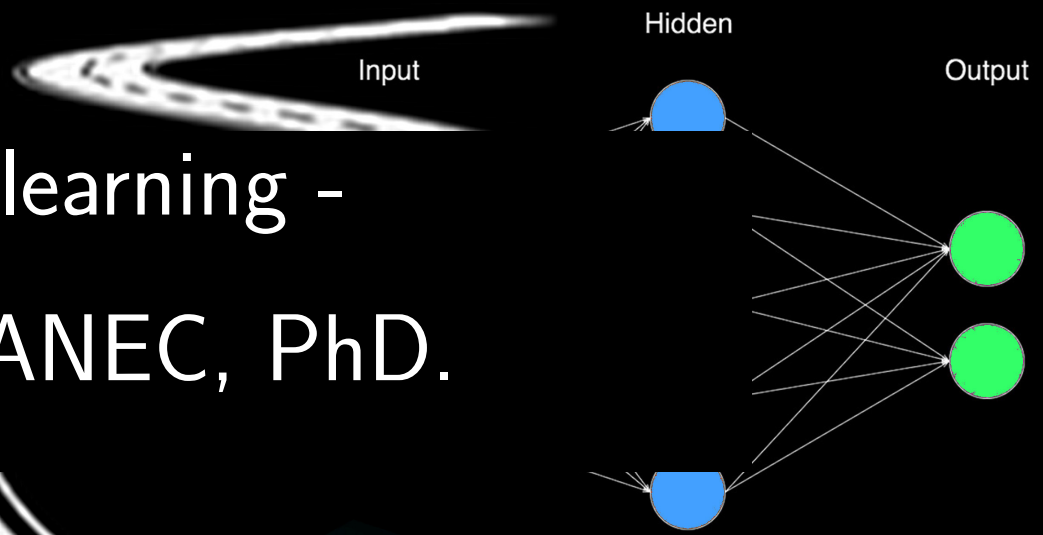
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\left[r_{t+1} + \lambda \max_a Q(s_{t+1}, a) - Q(s_t, a_t)\right]$$

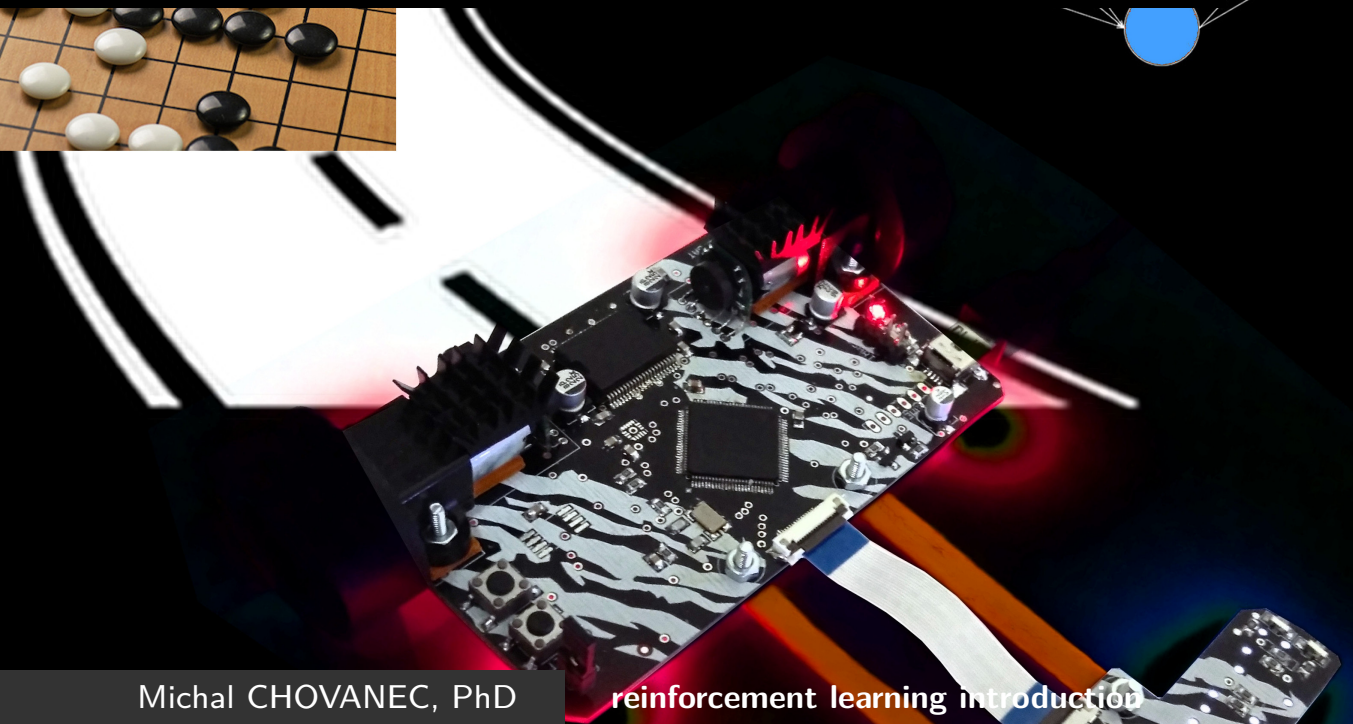
(The New Action Value = The Old Value) + The Learning Rate × (The New Information − the Old Information)



reinforcement learning - introduction

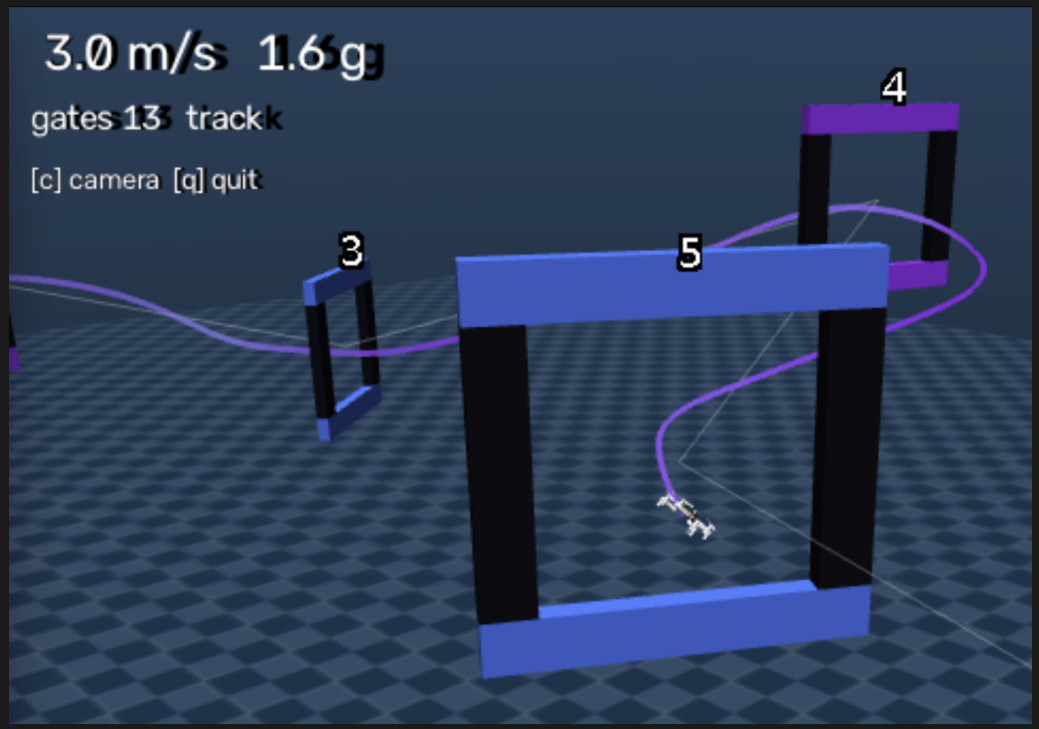
Michal CHOVANEC, PhD.





Overview

1. Agents - Principle
2. Example problem - Drone Racing
3. Policy Gradients - Advantage-Actor-Critic (A2C)
4. Practical Example - Lunar Lander
5. Cookbook and Literature

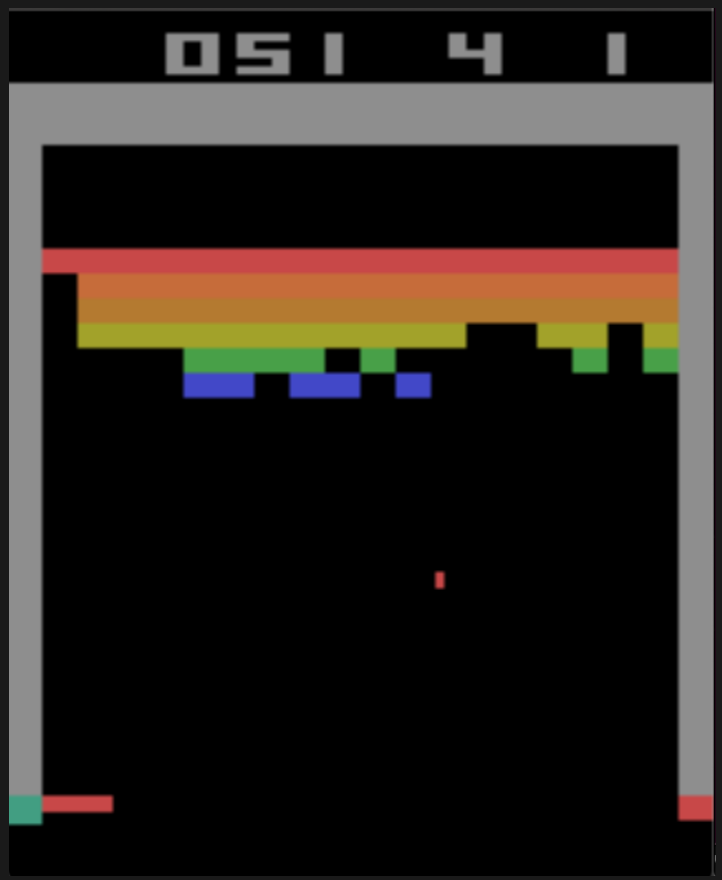


Reinforcement learning





Reinforcement learning - begining, 2013



- **Playing Atari with Deep Reinforcement Learning**
- human level gameplay from screen osbservation
- Deep Q networks (DQN)
- tiny model :
  8x8x16 - 4x4x16 - 256 - output

AlphaGo: Reinforcement Learning Beyond Human Play

- in 2016, AlphaGo defeated world champion Lee Sedol by **4–1**.
- combined deep neural networks, tree search, and reinforcement learning.
- through self-play, it learned strategies that went beyond simply imitating human games.



Why it matters: RL helped discover effective strategies in a vast decision space.

source: DeepMind, AlphaGo research project.

Autonomous Drone Racing: Beating Human Champions

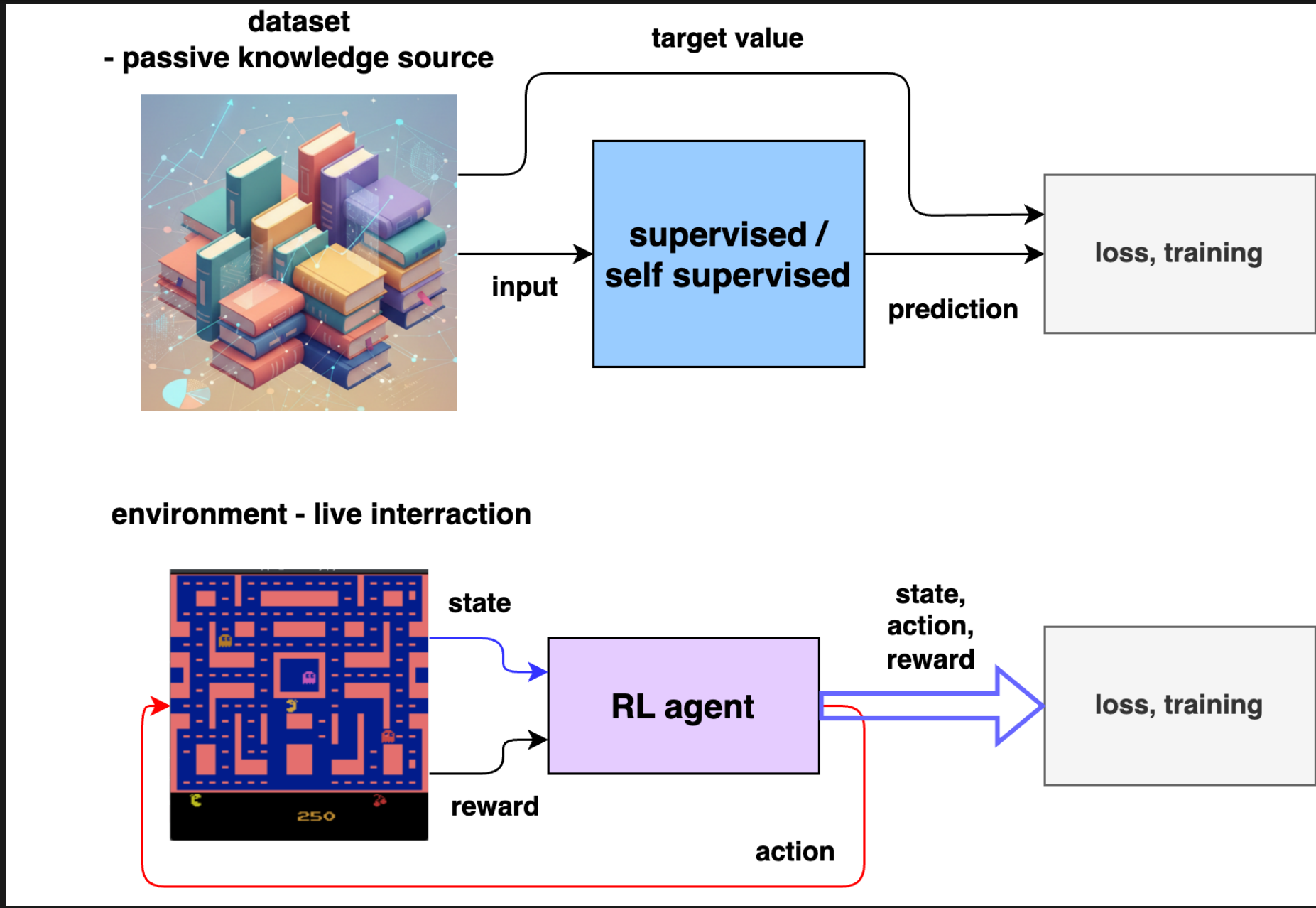
- deep reinforcement learning system learned to race a physical drone through gates.
- in head-to-head races, it won **15 out of 25** against three champion-level human pilots.
- its fastest race was **0.5 seconds faster** than the best human race time in the study.



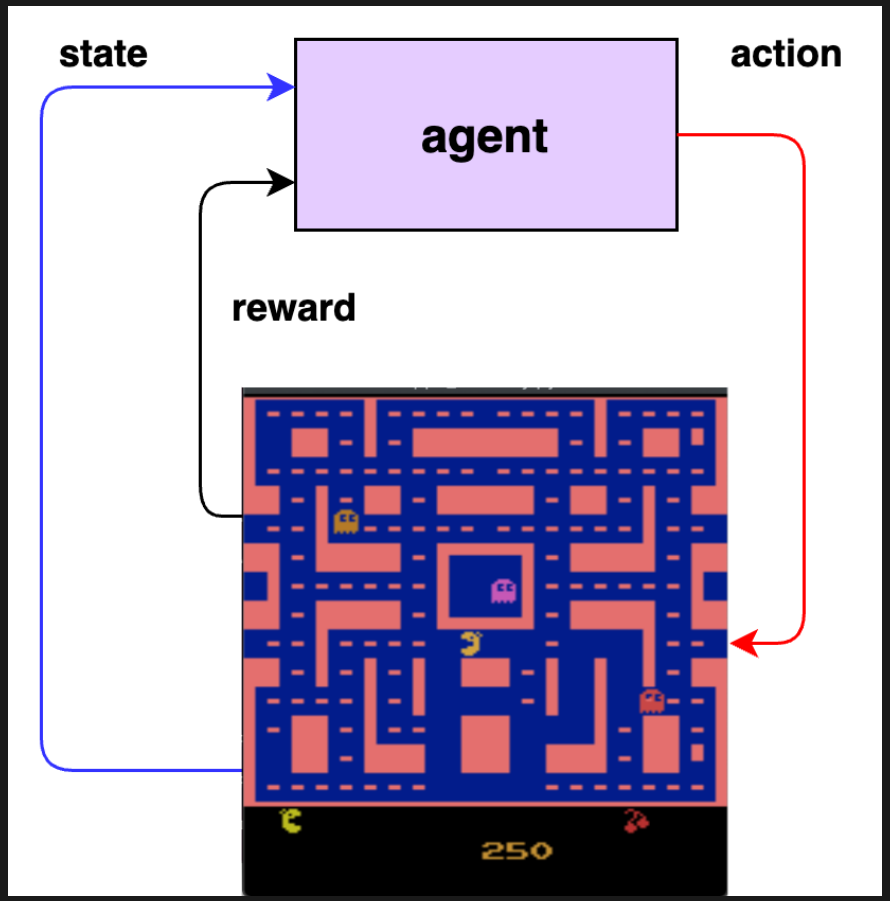
Why it matters: RL can learn fast, reactive control in the physical world, not just in games or simulations.

source: Kaufmann et al., Nature, 2023, "Champion-level drone racing using deep reinforcement learning".

Reinforcement learning - principle



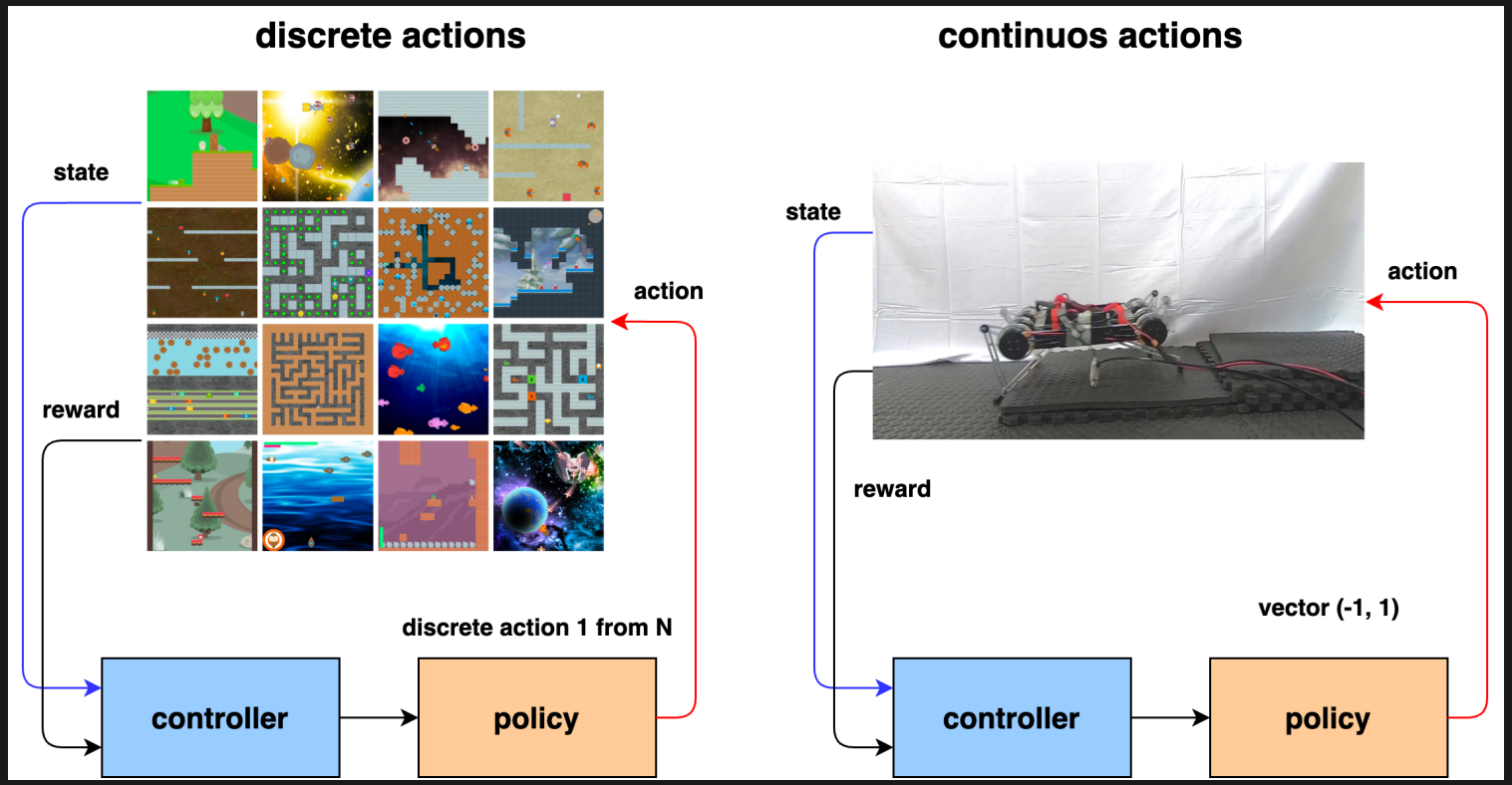
Reinforcement learning - principle



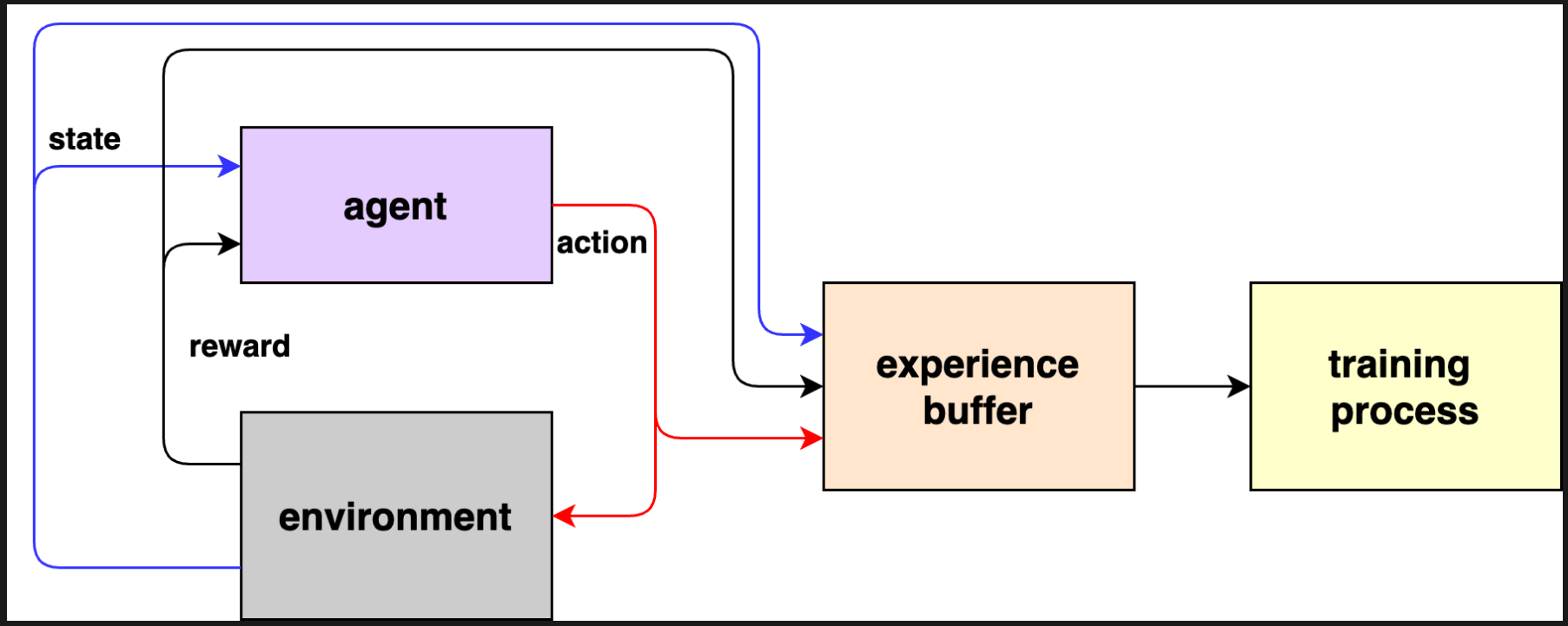
- **active environment interraction**
- obtain state
- select action
- exectute action
- learn from experiences

Action space

- discrete action space
  - keys, keypad
- continuous action space
  - motors, PWMs, steering, torque control



Reinforcement learning - training



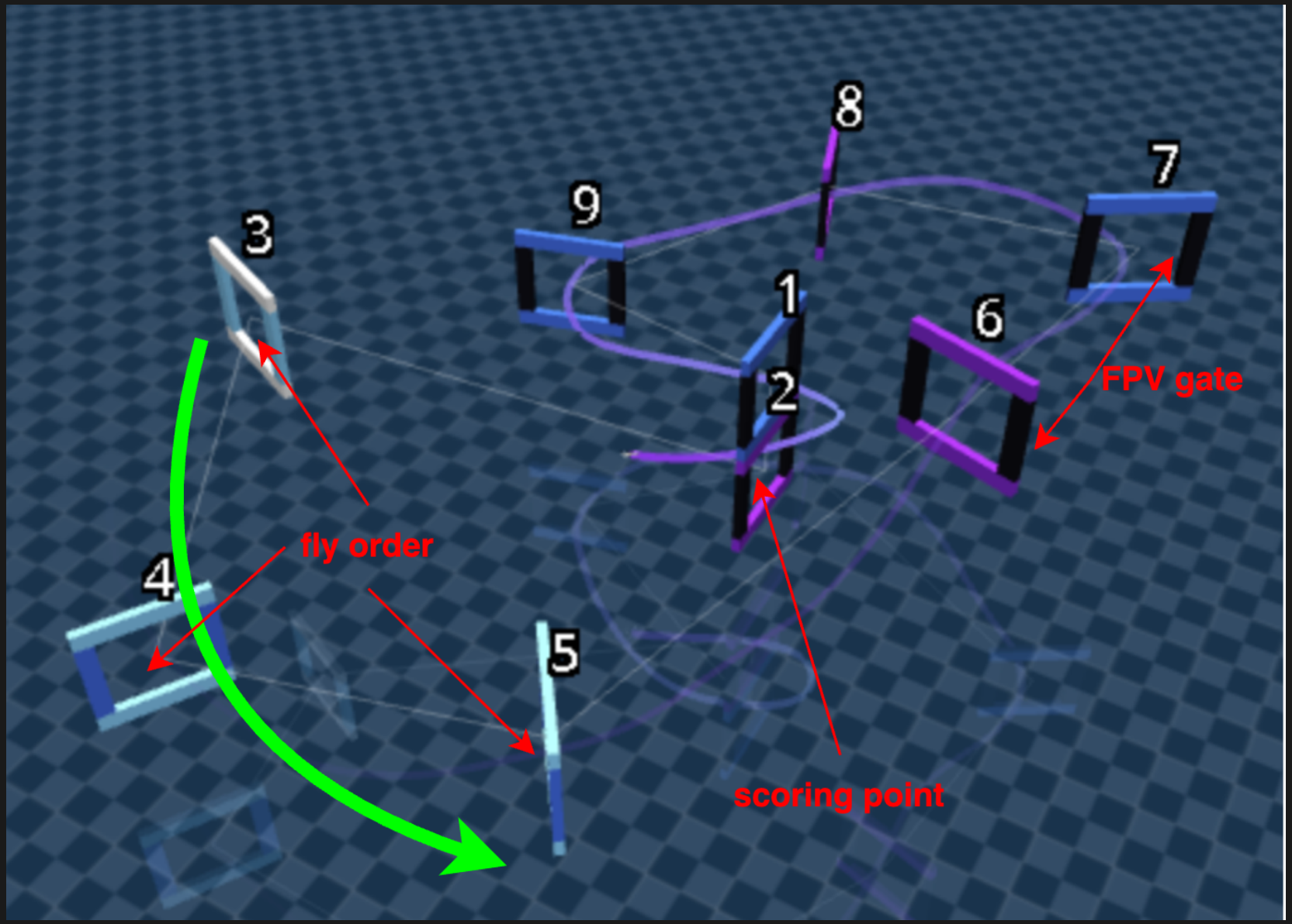
- obtain state
- select action
- exectute action
- learn from experiences

Common RL algorithms in practice

algorithm	year	best suited for	example / strength
DQN	2015	discrete actions	Atari games; learns values from replayed experience
A2C / A3C	2016	discrete or continuous actions	Atari and control; intuitive actor–critic baseline
PPO	2017	general-purpose policy learning	Robotics, locomotion and games; simple, robust baseline
SAC	2018	continuous control	Robot locomotion and manipulation; sample-efficient
TD3	2018	continuous control	MuJoCo locomotion; deterministic actor–critic

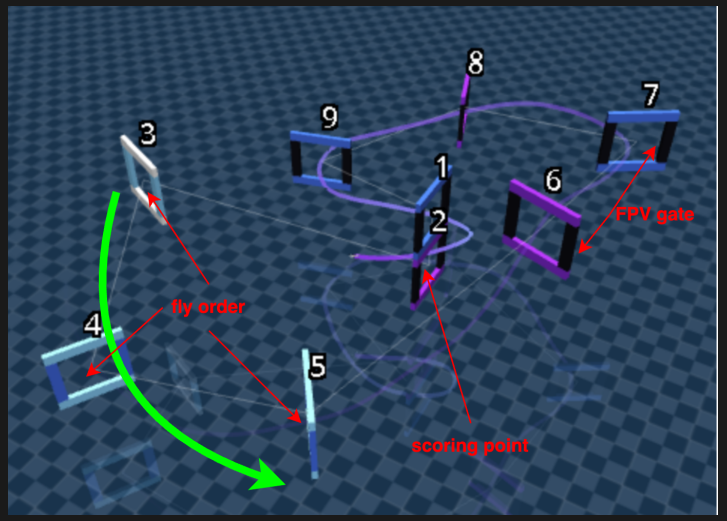
other important families: REINFORCE (1992), TRPO (2015), IMPALA (2018)

Example problem - drone racing



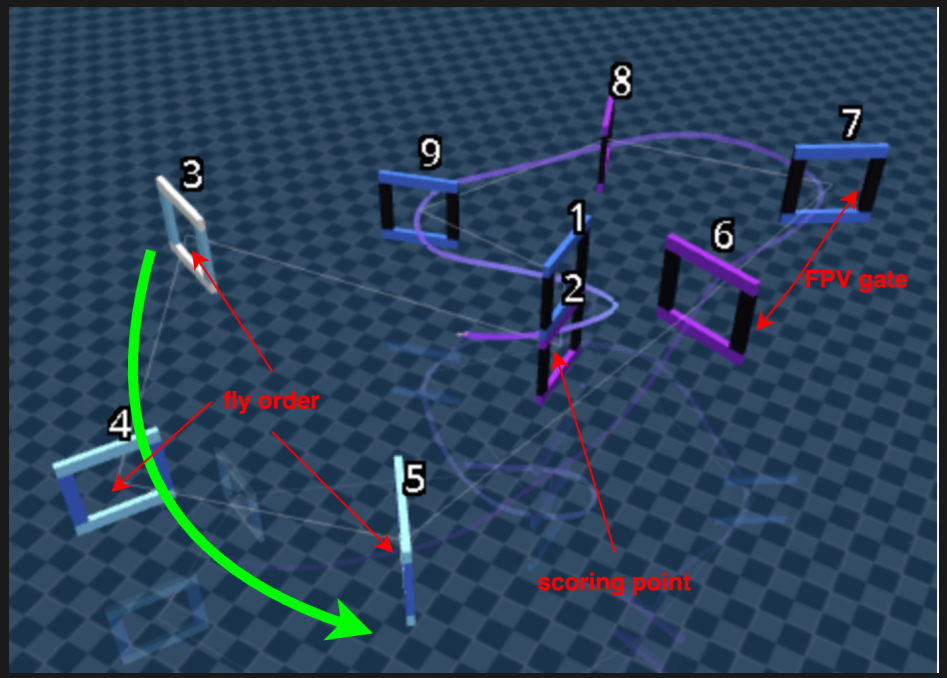
Example problem - drone racing

1. **environment design**
   - MuJoCo physics simulation
   - drone, gates and collisions
2. **state engineering**
   - what does the drone observe?
3. **reward engineering**
   - how do we define successful flight?
4. **agent selection**
   - PPO for continuous motor control
5. **training and evaluation**
   - learn, test and measure performance



fly through the gates in order

Environment design



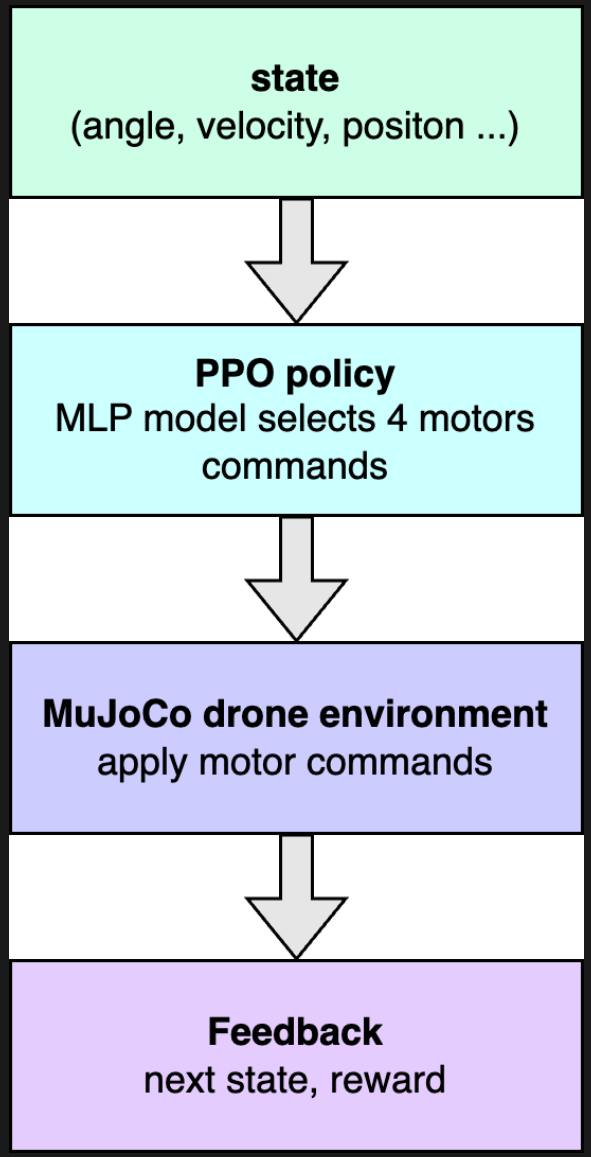
Simulation setup

- Crazyflie CF2X quadrotor
- MuJoCo physics engine
- 250 Hz physics simulation
- 50 Hz control frequency
- 5 physics steps per action
- 60 s maximum episode

Task

Fly through square gates in the prescribed order. Colliding with a gate frame terminates the episode.

Environment design



State Engineering

observation: what does the drone know?

- **attitude (4):** $\sin(\phi), \cos(\phi), \sin(\theta), \cos(\theta)$
- **angular velocity (3):** $\omega_x, \omega_y, \omega_z$
- **linear velocity (2):** velocity over ground
- **height (2):** distance above the floor and velocity
- **upcoming gates (36):** four 3D inner corners for each of three gates
- **nearest gate (12):** collision avoidance

observation vector

$$s_t \in \mathbb{R}^{59}, \qquad 11 + 3 \cdot 12 + 12 = 59$$

the observation is drone-centric: no absolute world position or world yaw is provided.

State Engineering - why these features?

- **attitude and angular velocity**
  - stabilize the drone and detect rotation.
- **velocity and height**
  - estimate motion and maintain safe flight.
- **relative gate geometry**
  - determine where the gate is and how it is oriented.
  - plan the approach using upcoming gates.

Design principle

give the policy task-relevant information in a consistent coordinate frame, not arbitrary world coordinates.

Reward Engineering

goal: pass gates in the correct order without crashing.

- gate passed: positive reward
- progress towards the next gate: shaping reward
- better gate alignment: shaping reward
- collision or crash: penalty

Reward function

$$r_t = \begin{cases} R_{\text{gate}}, & \text{if next gate passed,} \\ \Phi(s_t) - \Phi(s_{t-1}), & \text{otherwise} \end{cases}$$

$$r_t \leftarrow r_t - R_{\text{crash}} \not\Vdash_{\text{crash}}$$

Default values: $R_{\text{gate}} = 1$, $R_{\text{crash}} = 1$.

Reward Engineering - shaping

define a potential function:

$$\Phi(s) = -k_d d_{\text{gate}}(s) + k_a A_{\text{gate}}(s)$$

- d_{gate}: distance to the next gate centre
- A_{gate}: gate-alignment score
- k_d, k_a: shaping weights

Key idea

$$r_t^{\text{shape}} = \Phi(s_t) - \Phi(s_{t-1})$$

reward changes in progress and alignment, rather than rewarding their absolute values at every step.

with the potential reset handled consistently at gate transitions, this shaping avoids rewarding the agent simply for hovering near a gate.

Agent selection - PPO

why PPO?

- continuous action space: four motor commands
- stochastic policy enables exploration
- actor-critic architecture learns actions and values
- clipped objective limits incentives for large policy changes

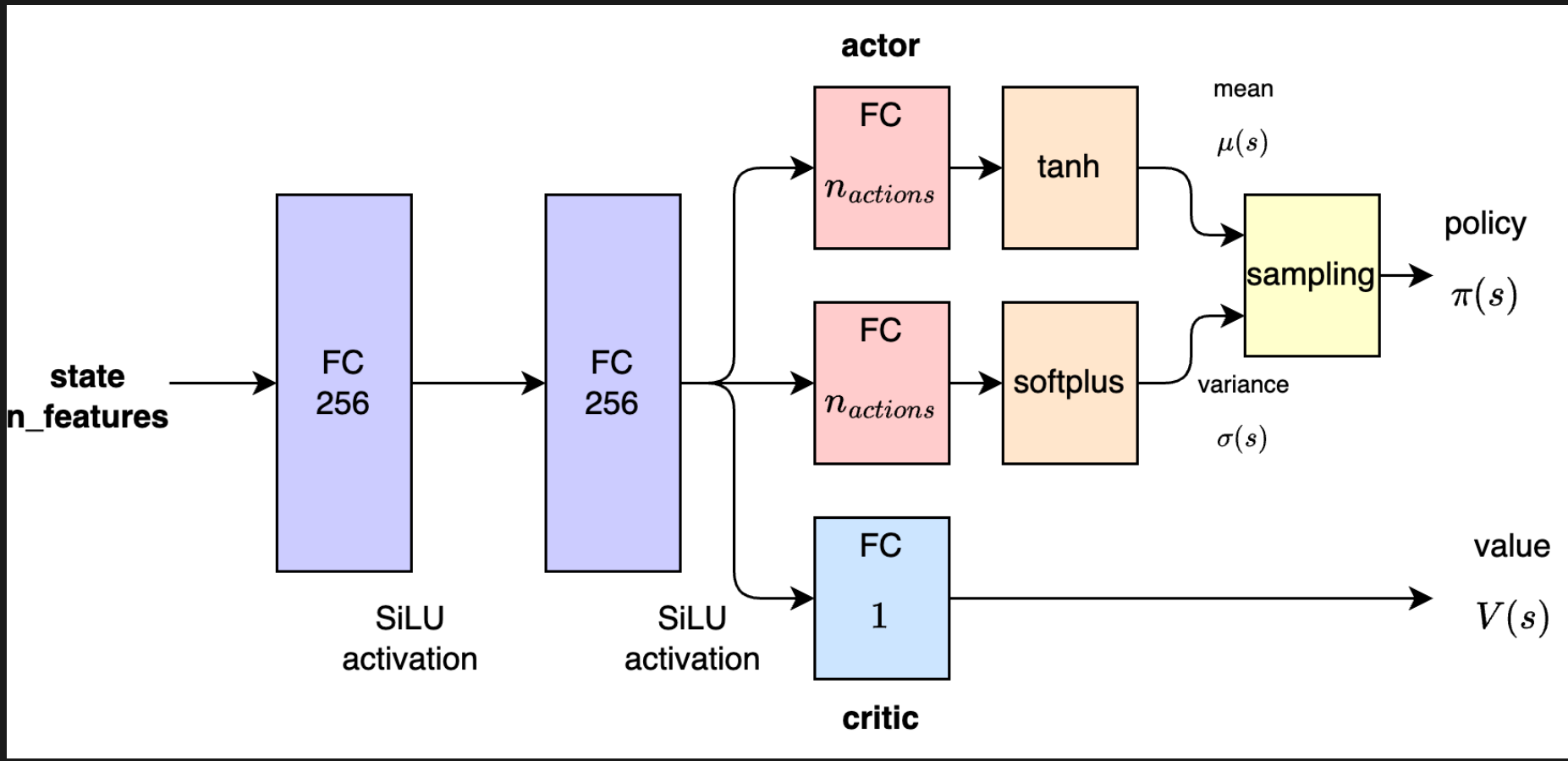
continuous policy

$$\pi_\theta(a|s) = \mathcal{N}\left(\mu_\theta(s), \text{diag}(\sigma^2_\theta(s))\right)$$

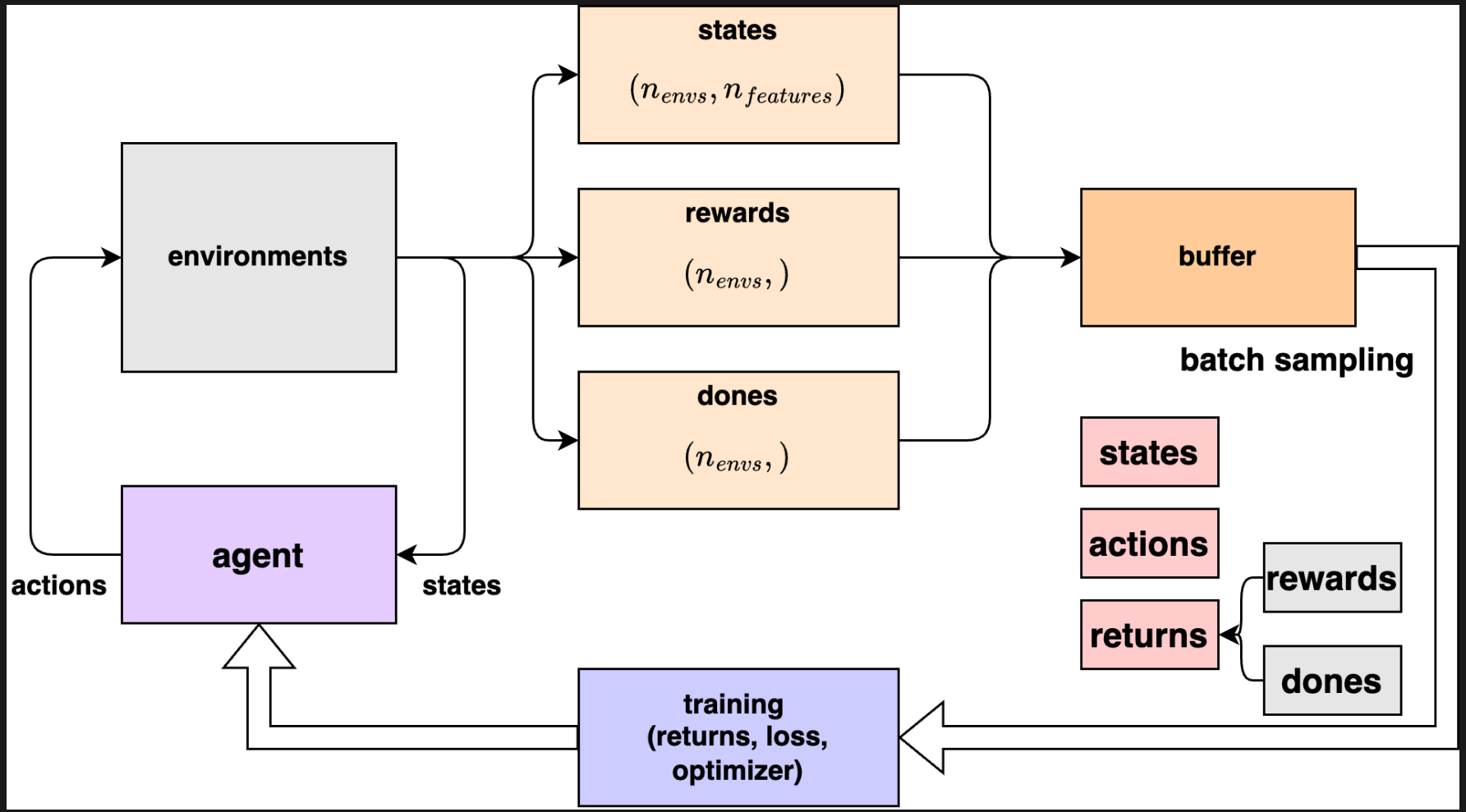
$$a_t = \mu_\theta(s_t) + \sigma_\theta(s_t) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

the model predicts the action mean, action variance and state value. The environment clips motor commands to $[-1, 1]^4$.

Agent selection - Model



PPO - learning from experience

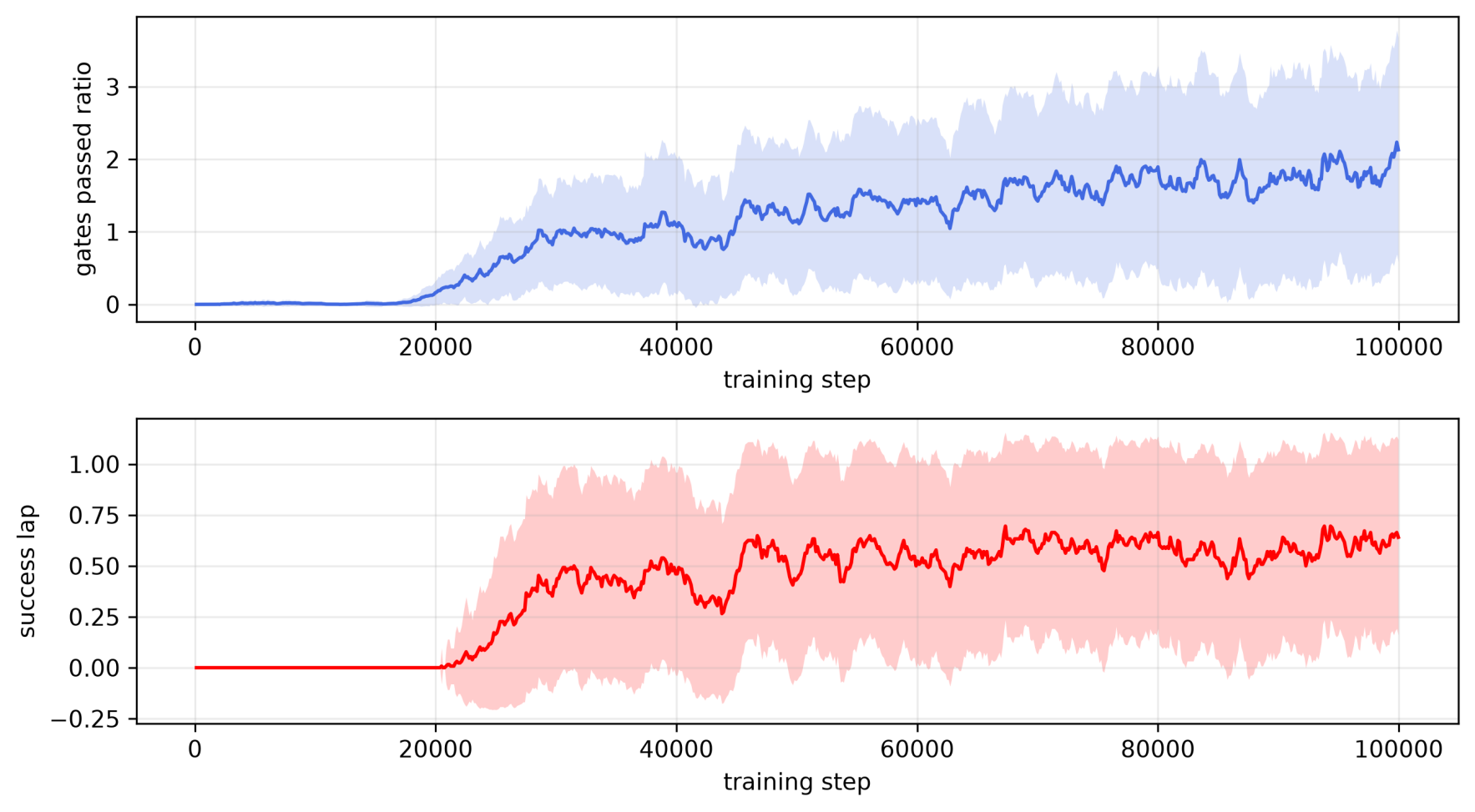


Evaluation - does the drone learn to race?

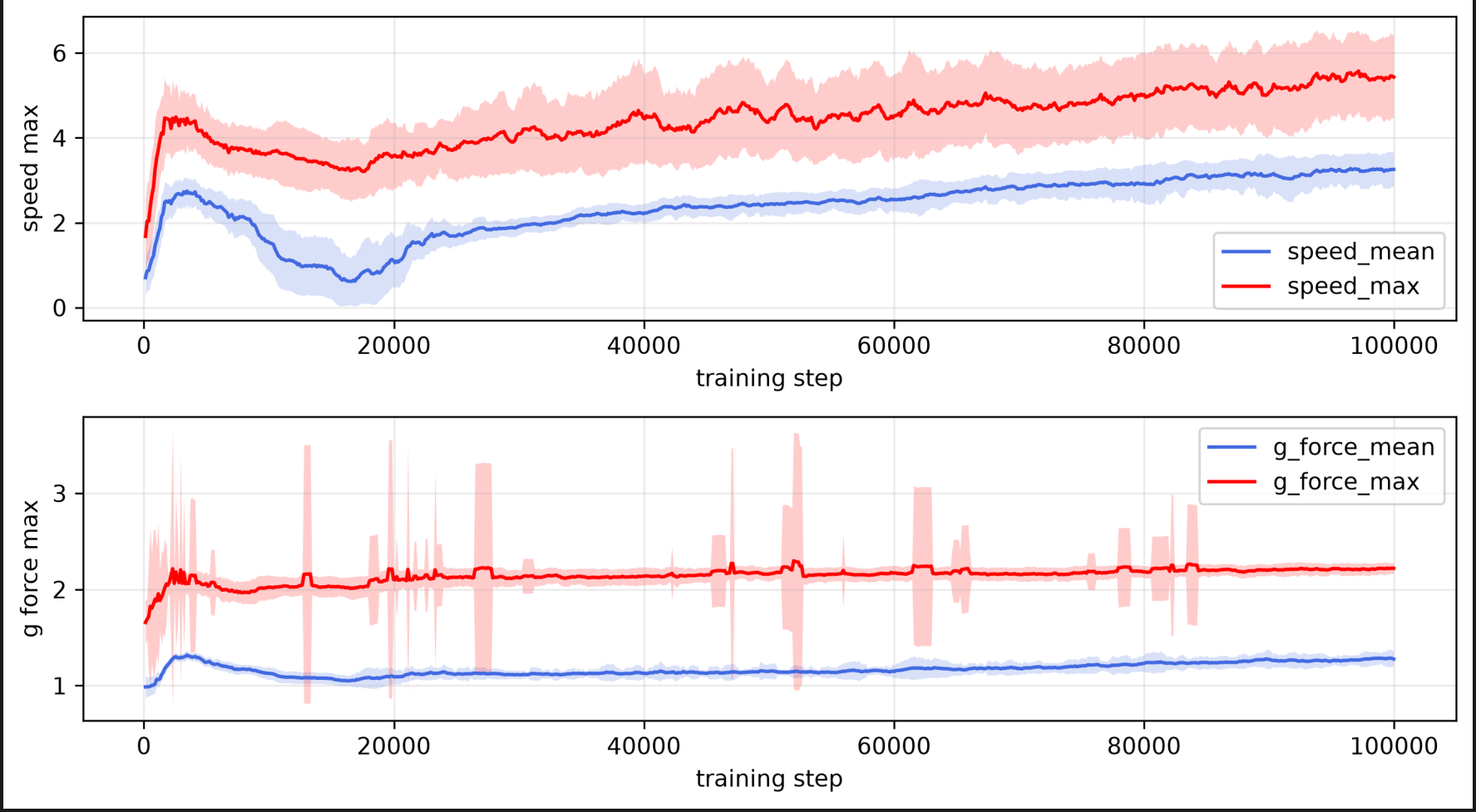
evaluation of the policy

- **episodic score**
- **gates passed:** mean number of successful crossings
- **flight speed and g-force:** mean and maximum speed
- **completion ratio:** gates passed / gates in course
- **crash rate:** fraction of episodes ending in a crash
- **generalization:** performance on unseen gate layouts

Passed gates

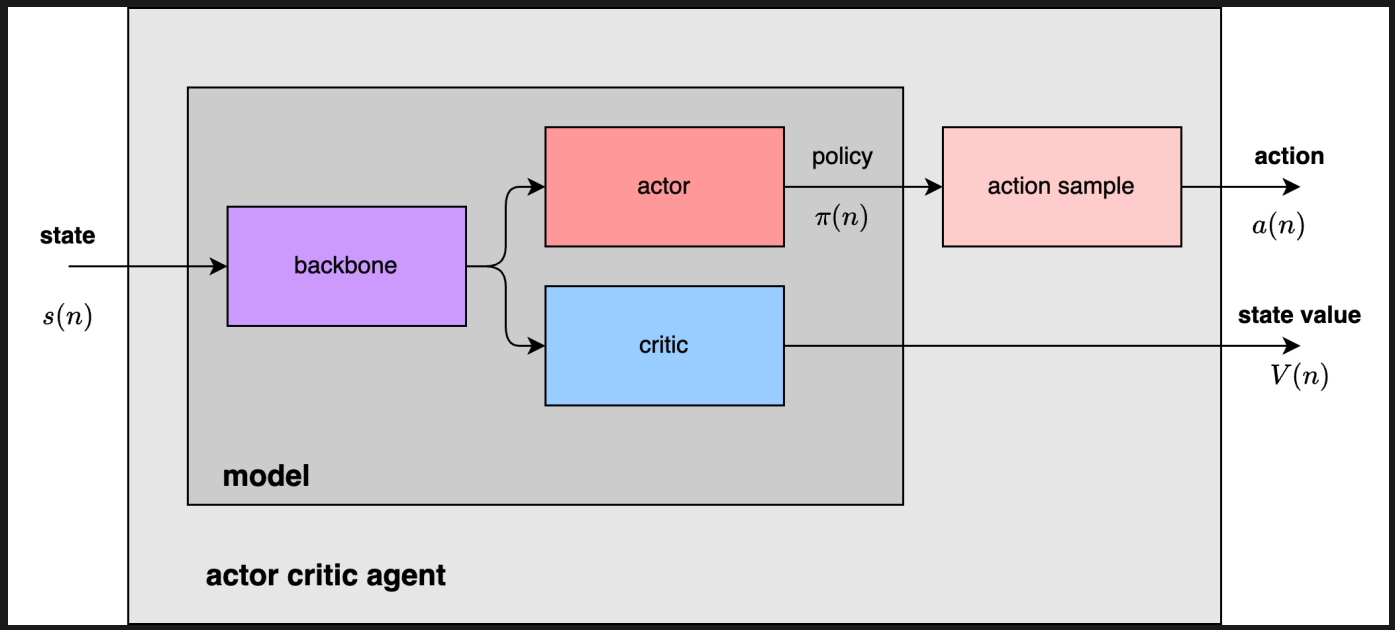


Speed and forces



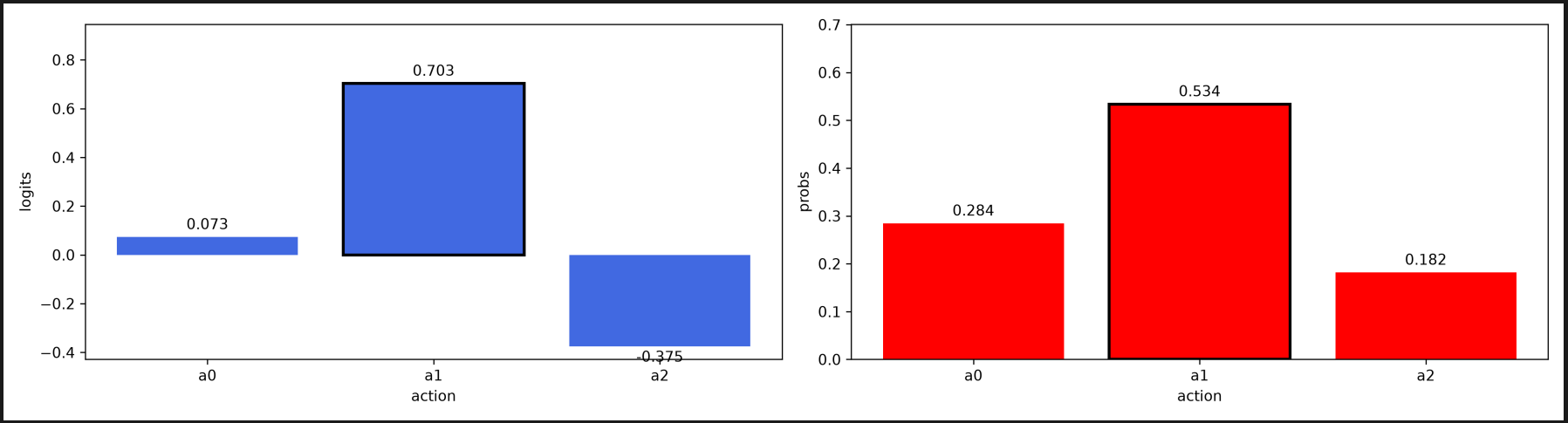
Policy Gradients, Advantage-Actor-Critic (A2C)

- **actor:** selects an action using policy $\pi_\theta(a|s)$
- **critic:** estimates expected future return $V_\phi(s)$
- **advantage:** tells the actor whether an action was better than expected



A2C is the simpler learning example; PPO builds on similar ideas and adds a more conservative policy update

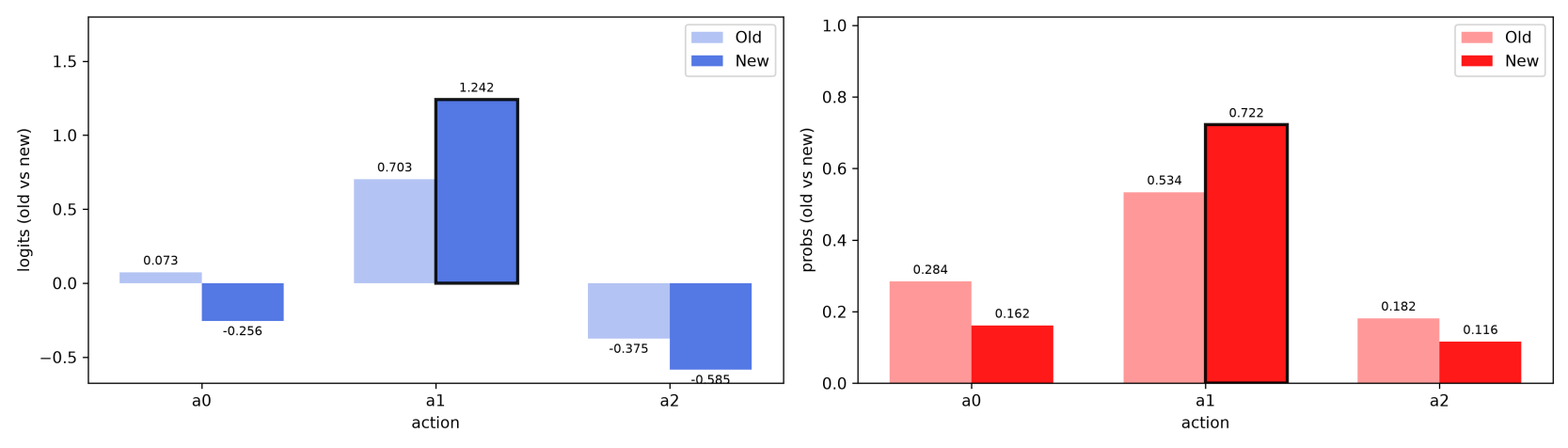
Action sampling



1. **policy output:** $f_\theta(s)$
2. **action probabilities:** $\pi_\theta(a|s) = \text{softmax}(f_\theta(s))$
3. **sample an action:** $a \sim \pi_\theta(\cdot|s)$

policy defines a probability distribution over available actions
during training, sampling encourages exploration;
during evaluation, we can select the highest-probability action

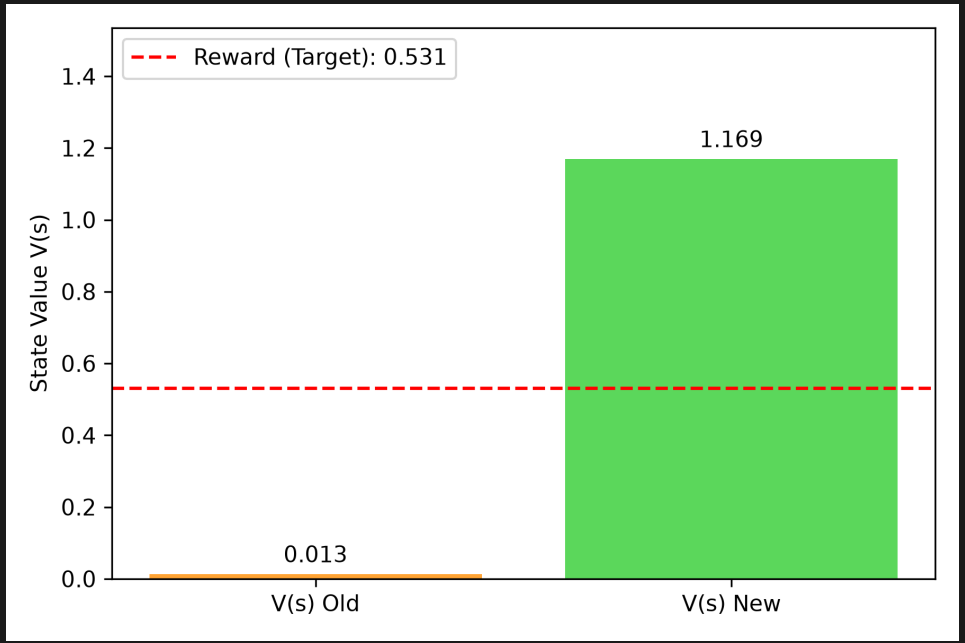
Actor training



1. **advantage:** $A(s, a) \approx R - V_\phi(s)$
2. **policy loss:** $\mathcal{L}_\theta = -A(s, a) \log \pi_\theta(a|s)$

positive advantage increases the probability of the selected action;
negative advantage decreases it
in practice, the return estimate may include multiple rewards and bootstrapping from the critic

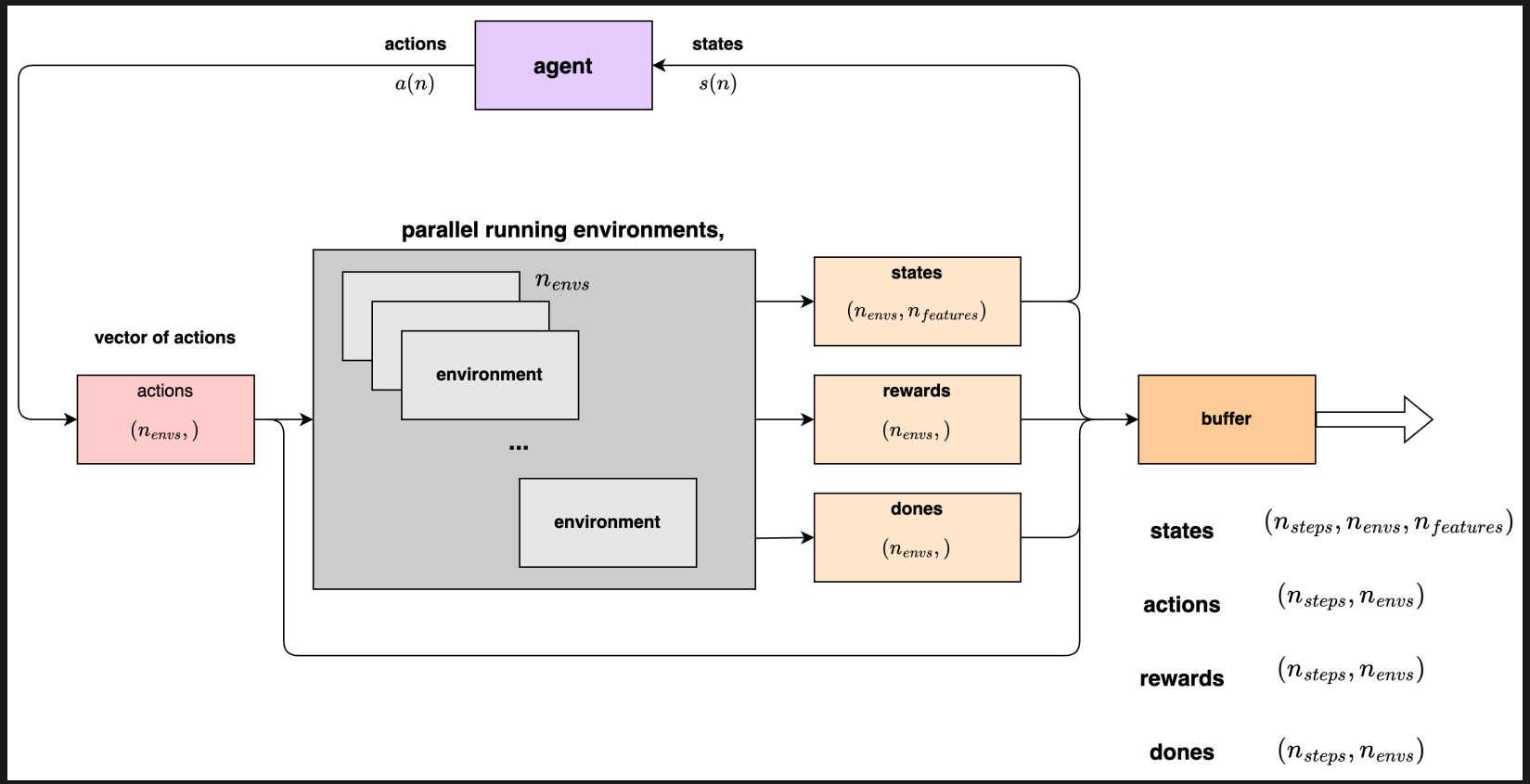
Critic training



1. **target return:** R_t
2. **value prediction:** $V_\phi(s_t)$
3. **critic loss:** $\mathcal{L}_\phi = \left(V_\phi(s_t) - R_t\right)^2$

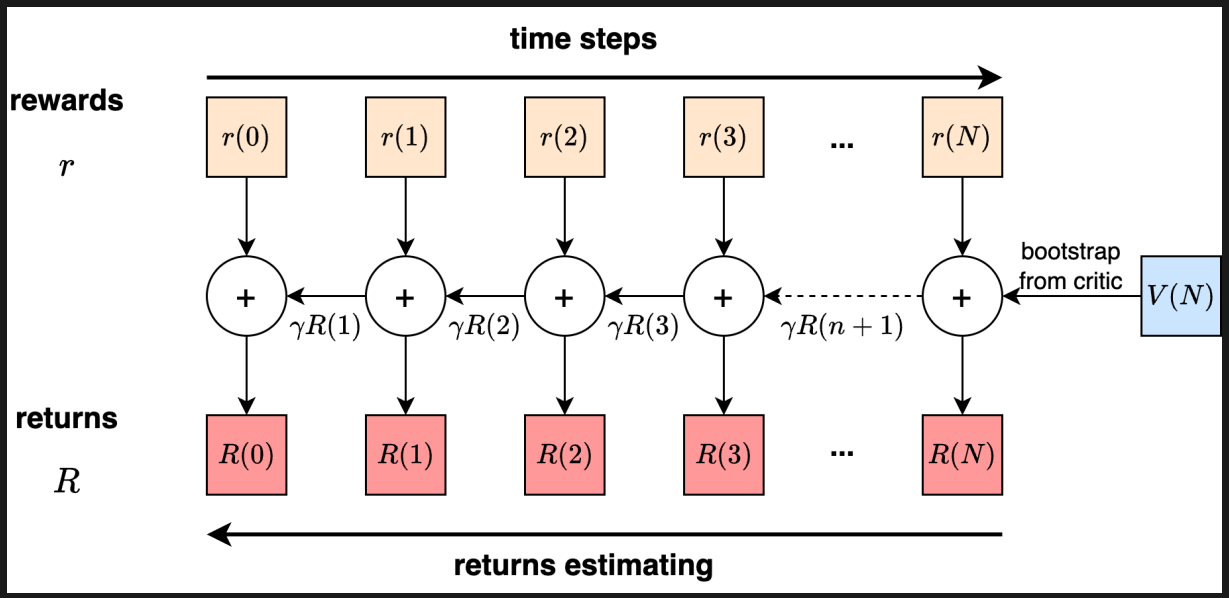
the critic learns to predict future discounted reward
better value estimates provide a less noisy baseline for the actor's policy-gradient updates

A2C training



- collect experience by interacting with the environment
- store states, actions, rewards and termination flags
- estimate returns and advantages for the collected rollout

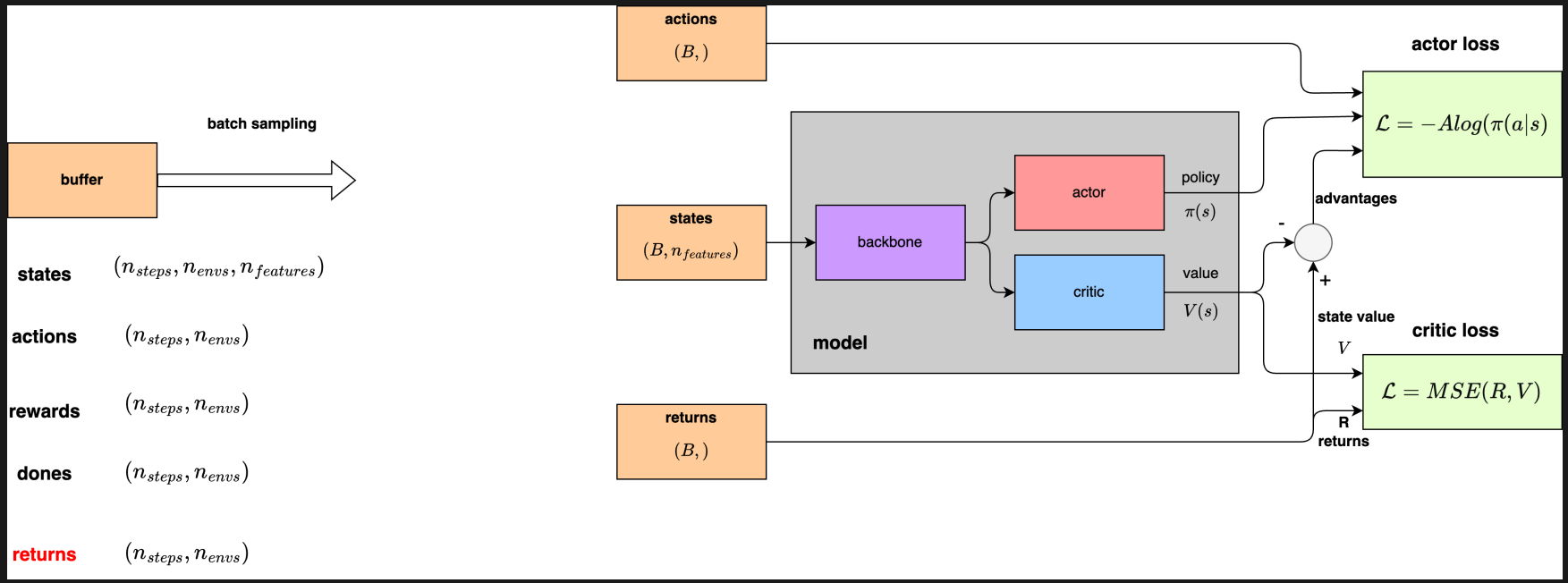
A2C training



- estimate **returns** and **advantages**
- update the **actor** using the estimated advantages
- update the **critic** by minimizing value prediction error
- combine policy loss, value loss and an entropy bonus

both networks learn from the same experience, but optimize different objectives

A2C training



Main takeaway

agent learns by repeatedly collecting experience, estimating which actions were good, and updating its policy

Cookbook - choose an environment

- **beginner: LunarLander**
  - small state vector, discrete actions, clear objective
  - ideal for understanding A2C and PPO
- **classic games: Atari**
  - Breakout, Space Invaders, Pac-Man
  - practical introduction to image-based RL
- **first-person games: ViZDoom**
  - start with `VizdoomBasic`
  - then try more complex scenarios and sparse rewards
- **robotics: MuJoCo**
  - `Ant`: locomotion with multiple actuators
  - `HalfCheetah`: forward running and speed control

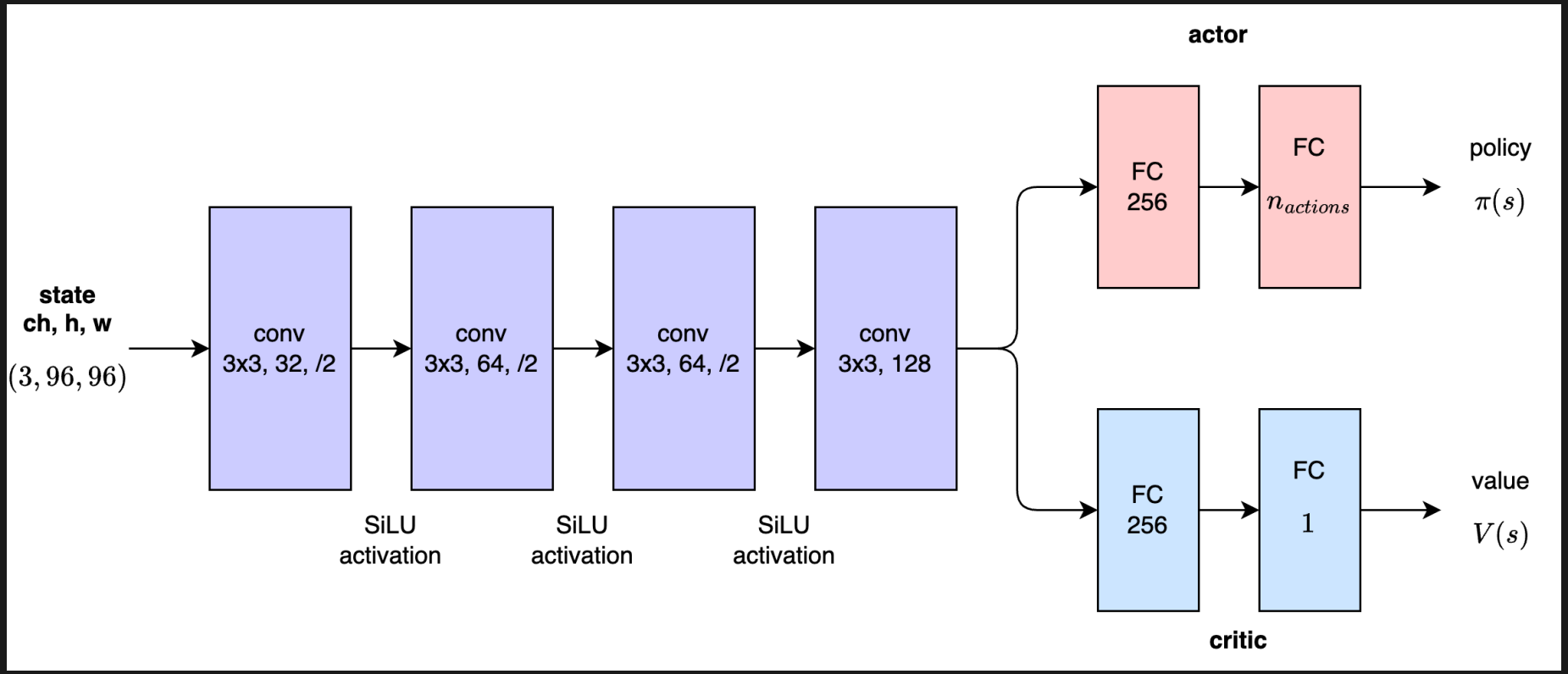
Official resources: LunarLander, Atari, ViZDoom, MuJoCo.

Cookbook - visual observations

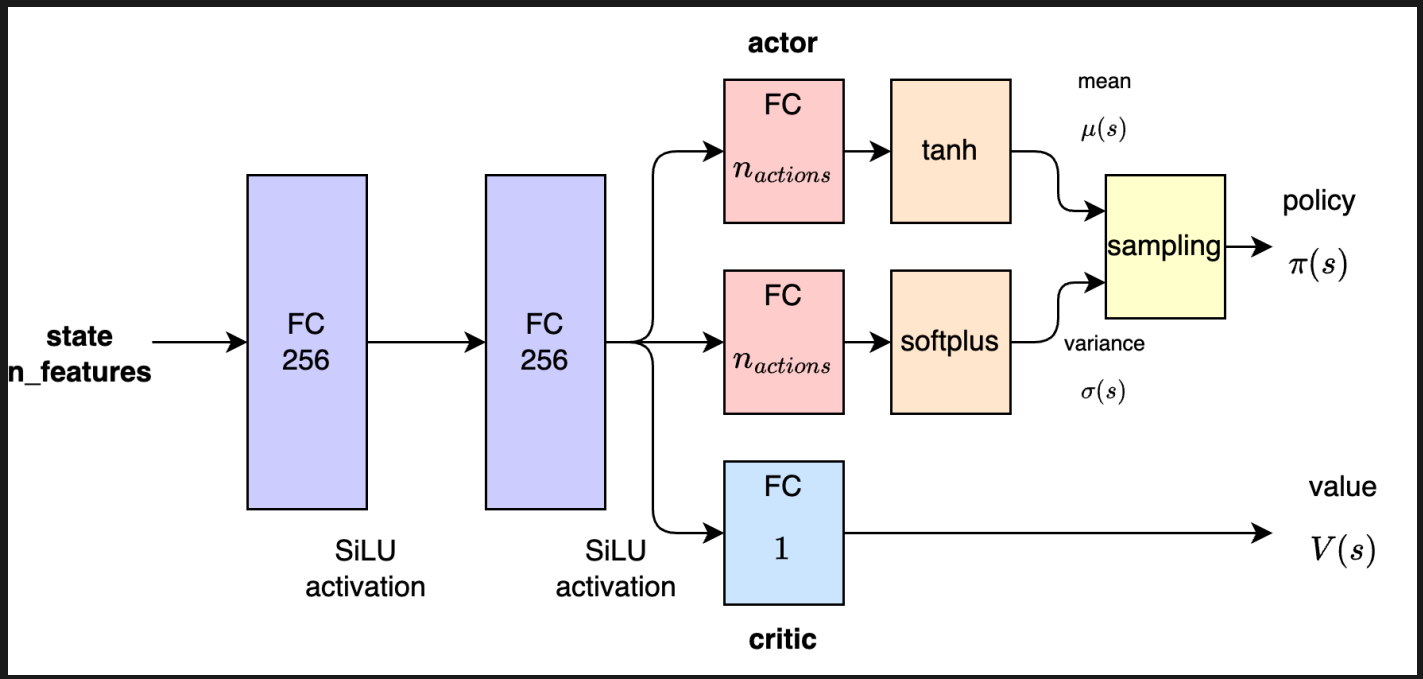
do not blindly copy the original DQN architecture

- **input preprocessing**
  - grayscale images to reduce input channels
  - stack 4 consecutive frames to capture motion
  - resize to preserve the information needed by the task
- **reasonable starting resolutions**
  - Atari: 96×96 as an experimental alternative
  - ViZDoom: 128×64 for a wide field of view
- **CNN feature extractor**
  - start with 3×3 strided convolutions and 4 layers
  - flatten the feature map and pass it to an MLP

CNN model example (Atari, Doom ...)



MLP model example (Robots, Drones ...)



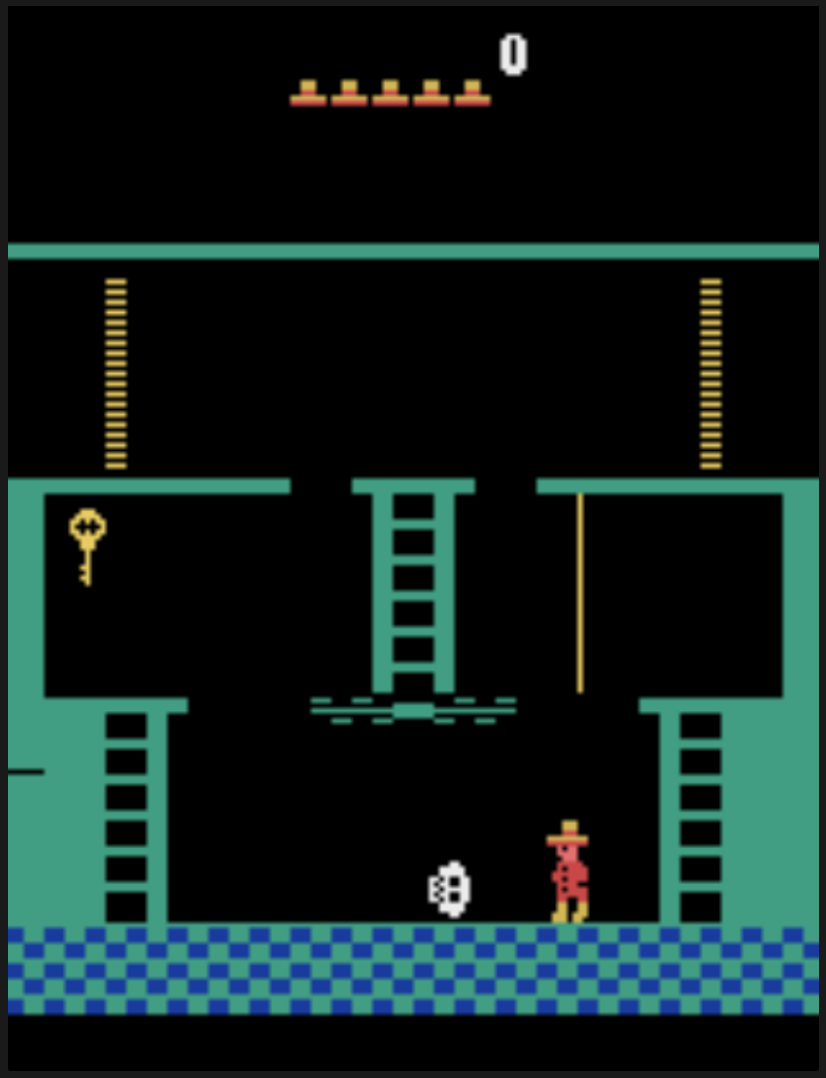
main challenges in reinforcement learning

- **Sample efficiency**
  - learning may require millions of environment interactions
  - real robots are slow, expensive, and can be damaged
- **Exploration and sparse rewards**
  - how can an agent discover useful behaviour when success is rare?
  - poor exploration can leave the policy stuck in local solutions
- **Reward design and alignment**
  - proxy reward may be optimized without achieving the real goal
  - speed, safety, energy, and task completion may conflict
- **Generalization and robustness**
  - policy that succeeds in training may fail in new conditions
  - sensor noise, unseen layouts, and changed dynamics matter

Core question

can we learn behaviour that is efficient, reliable, and useful beyond the training environment?

Montezuma's Revenge: exploration with self-supervised distillation



Montezuma's Revenge: platforms, ladders, hazards, and keys

The problem

- rewards are sparse: progress may produce no immediate feedback
- the agent must discover unfamiliar states and long action sequences

Self-supervised Network Distillation (SND)

- predictor learns to match features from a target network
- prediction error provides an intrinsic novelty signal
- the agent is encouraged to visit states it does not yet represent well

Key idea

use novelty-driven exploration to discover rewarding behavior when the game itself provides little feedback

M. Pecháč, M. Chovanec, I. Farkaš, *Self-supervised network distillation: an effective approach to exploration in sparse reward environments*, Neurocomputing, 2024. doi:10.1016/j.neucom.2024.128033

Literature - three papers to remember

1. DeepMind – the DQN milestone
V. Mnih et al.,
Human-level control through deep reinforcement learning
Nature, 2015.
nature.com/articles/nature14236

2. ETH Zurich – champion-level drone racing
E. Kaufmann et al.,
Champion-level drone racing using deep reinforcement learning
Nature, 2023.
nature.com/articles/s41586-023-06419-4

3. Self-supervised Network Distillation (SND)
M. Pecháč, M. Chovanec, I. Farkaš,
Self-supervised network distillation: an effective approach to exploration in sparse reward environments
Neurocomputing, 2024.
doi:10.1016/j.neucom.2024.128033

Literature - further reading

Actor–Critic and asynchronous learning
V. Mnih et al.,
Asynchronous Methods for Deep Reinforcement Learning
ICML, 2016.
PMLR: A3C paper

Practical policy optimization
J. Schulman et al.,
Proximal Policy Optimization Algorithms
arXiv, 2017.
arXiv:1707.06347

Continuous control and sample efficiency
T. Haarnoja et al.,
Soft Actor-Critic Algorithms and Applications
arXiv, 2018.
arXiv:1812.05905

Theory: PPO builds on policy-gradient and trust-region ideas. Read the PPO paper for the clipped objective, surrogate loss and empirical comparisons.

policy gradient: learning from samples

- our goal is to **maximize** the expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)].$$

- in practice, **we do not know the exact distribution** of all possible trajectories
- instead, we collect trajectories by running the policy and use a **Monte Carlo** estimate:

$$J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} R(\tau_i).$$

- to improve the policy, we need the gradient $\nabla_\theta J(\theta)$

policy gradient: the log trick

we only **have sampled trajectories**, so we want a gradient we can estimate from those samples

$$\begin{aligned}\nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] \\ &= \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\nabla_\theta \log P(\tau;\theta)\right]\end{aligned}$$

- **the key identity is**

$$\nabla_\theta P(\tau;\theta) = P(\tau;\theta)\nabla_\theta \log P(\tau;\theta)$$

- it turns the gradient of a probability into a log-probability gradient that we can evaluate for sampled trajectories
- since the environment's dynamics do not depend on θ,

$$\nabla_\theta \log P(\tau;\theta) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N} R(\tau_i)\sum_t \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t})$$

REINFORCE: learning from discounted returns

for each time step, define the **discounted return**:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$$

return measures the rewards collected after taking action a_t in state s_t, with future rewards discounted by γ.

REINFORCE gradient estimator is

$$\hat{g} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T_i-1} G_t^{(i)} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}).$$

- collect trajectories using the current policy
- compute the discounted return G_t for each action
- increase the probability of actions followed by high returns; decrease it for actions followed by low returns

REINFORCE: the problem of high variance

return G_t is a **noisy estimate of how good an action was**

- **different outcomes:** similar states and actions can lead to very different future rewards
- **delayed rewards:** an outcome may depend on many later actions
- **noisy updates:** different sampled trajectories can suggest very different parameter updates

consequence: REINFORCE gives an unbiased gradient estimate, but its high variance can make learning slow and unstable

Reducing variance: critic and advantages

critic estimates how much **future reward we expect from a state** under the current policy :

$$V^{\pi}(s_t) = \mathbb{E}_{\pi}[G_t \mid s_t].$$

instead of using the return directly, we compute the **advantage**:

$$\hat{A}_t = G_t - V_{\phi}(s_t).$$

actor update becomes

$$\hat{g} = \sum_t \hat{A}_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t).$$

- $\hat{A}_t > 0$: **the outcome was better than expected**; increase the action's probability
- $\hat{A}_t < 0$: **the outcome was worse than expected**; decrease the action's probability
- $\hat{A}_t \approx 0$: **the outcome was close to expectation**; make little change.

Q&A

michal.nand@gmail.com
https://github.com/michalnand/lectures/